

🚀 Extra — Bullets, Lists, and the Frame Loop

Topic: Lists · Deleting Safely · Timers · Collisions · **Course:** Extra Worksheet · **Time:** about 60 minutes

Keep these words handy: **frame, list, copy, index, cooldown, collide, remove**

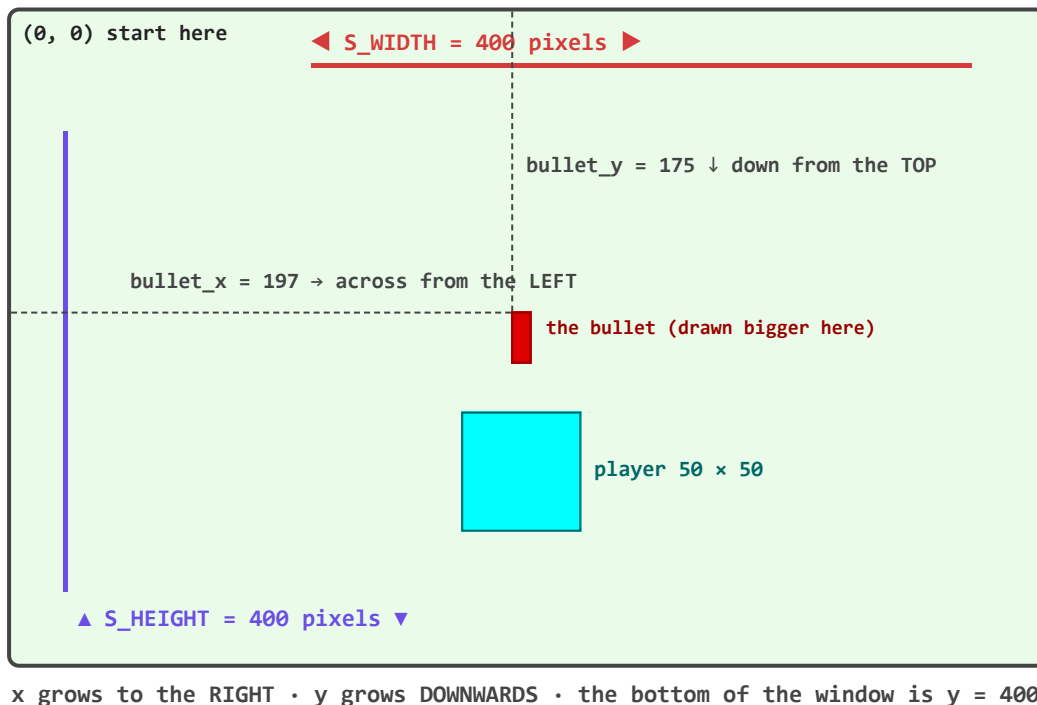
Today you gave your ship bullets. A bullet is a small list `[x, y]` living inside a bigger list `bullets`. Everything hard about bullets is really about lists: adding to them, walking them, and deleting from them safely. This sheet is reading and tracing on paper first, then two builds in your own game.

HOW TO USE THIS SHEET

1. Read on paper first. Do not open the IDE until section 8.
2. Every question has small steps under it. Do them in order.
3. Stuck? Read the yellow box. It points at where to look — never the answer.
4. Write something in every box. A wrong answer you can talk about beats an empty box.

1 · The Map of the Window 🗺️

Before any code: every number in your game is a distance in **pixels**, measured from the top-left corner.



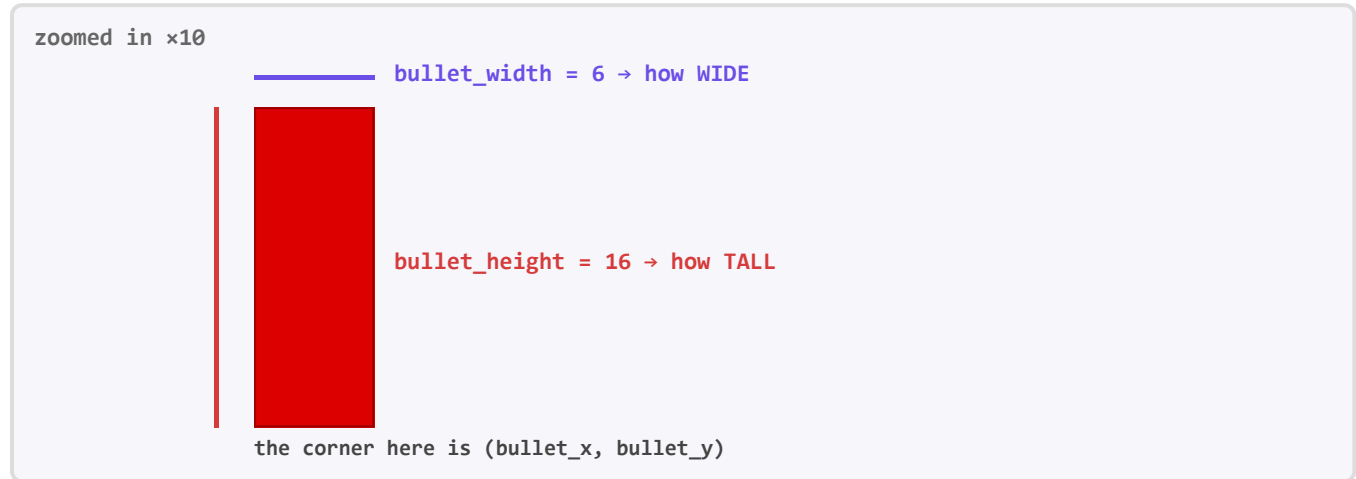
In one sentence each: what is `S_WIDTH` , and what is `S_HEIGHT` ?

A bullet sits at `[197, 20]` . Describe where it is on the screen.

Which is bigger for something near the top of the screen — a big `y` or a small `y` ?

2 · The Size of One Bullet

`bullet_x` and `bullet_y` say **where** the bullet is. `bullet_width` and `bullet_height` say **how big** it is. Four different jobs, four different numbers.



THE LINE THAT USES ALL FOUR

```
pygame.draw.rect(screen, bullet_colour, (bullet[0], bullet[1], bullet_width, bullet_height))
```

Position first, size second. Always in that order.

Your bullet is 6 wide and 16 tall. Draw the shape you expect — tall and thin, or short and fat?

Which two of these four numbers change while the bullet flies up the screen?

A player at `player_x = 380` is 50 wide, in a window 400 wide. What is wrong, and which line in your game stops it?

3 · Predict 🧠

Read each block. Write what happens — just from reading.

```
bullet = [197, 300]
bullet_speed = 12

bullet[1] = bullet[1] - bullet_speed
print(bullet)
```

Step 1 — Which slot changed, and what does that slot mean?

Step 2 — What prints? Did the bullet move up or down?


```
bullets = []
bullets.append([197, 300])
bullets.append([197, 300])
bullets.append([197, 300])

print(len(bullets))
```

What prints? Why are they allowed to look identical?


```
bullet_cooldown = 300
bullet_lastshot = 0
now = 250

print(now - bullet_lastshot > bullet_cooldown)
```

Step 1 — Work out the left side of the `>` first.

Step 2 — True or False? So does a bullet get made?


```
bullets = [[0, 10], [0, 20], [0, 30], [0, 40]]

for bullet in bullets:
    bullets.remove(bullet)

print(bullets)
```

Careful — this is the trap from the lesson. What is left in the list?

4 · Trace the Cooldown

Your game runs 60 frames a second, so `now` jumps about 16 each frame. You are holding SPACE the whole time.

`bullet_cooldown = 300`

HOW TO FILL EACH ROW

1. Work out `now - bullet_lastshot` .
2. Is that bigger than 300? Write yes or no.
3. If yes, a bullet is made and `bullet_lastshot` becomes `now` .
4. If no, `bullet_lastshot` does not change. Carry it down to the next row.

now	bullet_lastshot	now - lastshot	bigger than 300?	bullet made?
0	0	0	no	no
160				
320				
480				
640				
800				

Holding SPACE for 3 whole seconds — how many bullets with a cooldown of 300?

And with no cooldown at all, at 60 frames a second?

STUCK?

3 seconds is 3000 milliseconds. How many 300s fit inside 3000?

5 · Trace the Bullets List

One bullet is fired, then the game runs three more frames. `bullet_speed = 12`. Bullets are deleted when `y` goes below 0.

Fill in the list after each frame.

frame	what happens	bullets list after the frame	len(bullets)
1	SPACE pressed, bullet made at $y = 300$	[[197, 300]]	1
2	move loop runs		
3	move loop runs, SPACE pressed again (cooldown allows it)		
4	move loop runs		

The new bullet in frame 3 starts at $y = 300$ again. Why not at 276 like the first one?

6 · Spot the Bug 🐛

Every one of these is a real mistake from a real shooter game. Work the steps for each.

BUG METHOD — USE IT EVERY TIME

1. Say what this code is **supposed** to do.
2. Read it one line at a time and say what it **actually** does.
3. Circle the exact line where those two stop matching.
4. Write the fixed code.
5. Say why your fix works.

Bug A — Bullets should disappear off the top. Instead about half of them stay in the list forever.

```
for bullet in bullets:
    if bullet[1] < 0:
        bullets.remove(bullet)
```

Hint: four bullets go in. Count how many the loop actually looks at.

Write the fixed code:

Why was it wrong? Why does your fix work?

Bug B — Holding SPACE gives a solid wall of bullets. The cooldown seems to do nothing.

```
if keys[pygame.K_SPACE] and now - bullet_lastshot > bullet_cooldown:
    bullet_x = player_x + (player_width // 2) - (bullet_width // 2)
    bullet_y = player_y
    bullets.append([bullet_x, bullet_y])
```

Hint: something must change after a shot, or the sum stays true forever.

Write the fixed code:

Why was it wrong? Why does your fix work?

Bug C — Bullets come out of the ship and fall to the bottom of the screen.

```
for bullet in bullets:
    bullet[1] = bullet[1] + bullet_speed
```

Hint: in pygame, y = 0 is the top.

Write the fixed code:

Why was it wrong? Why does your fix work?

Bug D — No bullets are ever visible, even though `len(bullets)` keeps going up.

```
for bullet in bullets:
    pygame.draw.rect(screen, bullet_colour, (bullet[0], bullet[1], bullet_width,
bullet_height))

screen.fill((152, 251, 152))
pygame.draw.rect(screen, CYAN, (player_x, player_y, player_width, player_height))
```

Hint: what does `screen.fill` do to everything drawn before it?

Write the fixed code:

Why was it wrong? Why does your fix work?

Bug E — This is from your own file. Three enemies cross the bottom on the same frame. The score only goes up by 2, and one enemy stays in the game invisibly — it can still end your game.

```
for enemy in enemies:
    if enemy[1] > S_HEIGHT:
        enemies.remove(enemy)
        score += 1
```

Hint: you have seen this exact shape already today.

Write the fixed code:

Why was it wrong? Why does your fix work?

7 · Explain the Code

Your own lines from today. Answer from reading only.

```
bullet_x = player_x + (player_width // 2) - (bullet_width // 2)
```

Step 1 — What does `player_x + (player_width // 2)` point at?

Step 2 — Why subtract half the bullet width after that?

Step 3 — What would the bullet look like if you deleted the whole subtraction?


```
for bullet in bullets[:]:  
    for enemy in enemies[:]:
```

Both lists carry `[:]`. Explain what would break if you removed it from just one of them.


```
if bullet_rect.colliderect(enemy_rect):  
    bullets.remove(bullet)  
    enemies.remove(enemy)  
    score = score + 10  
    break
```

Step 1 — What exactly does `break` stop?

Step 2 — Two enemies overlap each other and one bullet hits both. Without `break`, which line crashes, and what is the error called?


```
enemy_rect = pygame.Rect(enemy[0], enemy[1], enemy_width, enemy_height)
```

This line is inside the loop, not above it. Explain why it cannot be written once outside.

8 · Code It

Open `main.py` in the IDE at app.english-coding.co.uk. One task at a time. Run the game after every task.

Task 1 — Count your misses.

STEPS

1. Above the game loop, make a variable `misses = 0`.
2. Find the loop that deletes bullets leaving the top of the screen.
3. A bullet leaving the top means you missed. Add 1 to `misses` there.
4. Print `misses` at the same moment, so you can watch it in the console.

YOU KNOW IT WORKS WHEN

firing straight up with no enemy around makes the number climb, and hitting an enemy does not.

Task 2 — Show it on screen.

STEPS

1. You already build `score_surface` with `game_font.render`. Copy that pattern for misses.
2. Blit it under the score, not on top of it — change the second number in the position.
3. Run it. Both lines should be readable at the same time.

YOU KNOW IT WORKS WHEN

score and misses both show, and neither one overlaps the other.

Task 3 — Double barrel.

STEPS

1. Find the SPACE block where you append one bullet.
2. Append a second bullet in the same block, a little to the left.
3. Move the first one a little to the right, so they leave from both wings.
4. Run it. Both bullets should fly up together and both should be able to hit.

YOU KNOW IT WORKS WHEN

one press of SPACE puts two bullets on screen, and shooting one enemy with both does not crash the game.

STUCK ON ANY TASK?

Print the value you are unsure about — `print(misses)` , `print(len(bullets))` — run the game and read the console. If a change breaks everything, undo it and make a smaller change.

Write your plan here before you code.

9 · Explain Your Code 🎥

Record a short video on your phone explaining the code **you** wrote. Try to use these words: **list**, **copy**, **index**, **cooldown**, **collide**, **remove**.

1. Fire once and point at the line that made the bullet.
2. Point at the line that moves it and say why it is a minus.
3. Show your cooldown line and say what would happen without it.
4. Show one place you wrote `[ : ]` and explain what it protects you from.
5. Show your misses counter working.

Plan what you will say here before you record.

Submit 

Send this worksheet + your video or photo to teacher on KakaoTalk.